

25-Day Team Work Distribution and Execution Plan

Secure Token-Efficient Autonomous Coding Agent

Team size: 7

Project lead: Ghulam Qasim

Co-lead / integration lead: Ghulam Qadir

Submission window: 25 calendar days

Operating objective: Complete the project with clear ownership, minimal code collision, controlled integration, and enough time for testing, documentation, and demonstration.

1. Team Structure

1.1 Members and responsibilities

Person	Role	Primary ownership	Secondary responsibility
Ghulam Qasim	Team Lead / Product Owner	Scope, priorities, architecture decisions, acceptance criteria, final submission	Final review, stakeholder communication, conflict resolution
Ghulam Qadir	Co-Lead / Integration Lead	Repository structure, shared contracts, integration branch, CI, release coordination	Code review, daily technical coordination, backup for all modules
Azizullah + Yash + Jami	Backend and Task-Orchestration Engineers	Task API, run state machine, orchestrator, persistence	API documentation and backend tests; Azizullah is the workstream lead, Yash owns execution integration, and Jami owns model/context integration
Asif + Azizullah + Ghulam Qasim	Security and Policy Engineers	Security Gateway, policy engine, permissions, secret redaction, audit events	Threat model and security test cases; Asif is the technical owner, Azizullah integrates policy with orchestration, and Qasim provides final approval
Yash + Aamir + Asif	Sandbox and Tooling Engineers	Tool Router, sandbox runtime, filesystem/shell/network controls	Runtime observability and resource limits; Yash is the technical owner, Aamir validates behavior, and Asif approves security controls

Person	Role	Primary ownership	Secondary responsibility
Jami + Ghulam Qadir + Aamir	Context and Model Engineers	Repository index, retrieval/ranking, compression/cache, Nebius adapter/model routing	Token and latency measurements; Jami is the technical owner, Qadir integrates contracts, and Aamir evaluates results
All members	Validation, Evaluation, and Documentation Team	Validator/critic, test harness, baselines, metrics, user documentation	Demo preparation and final report; Aamir coordinates the workstream and Qasim approves the final submission

1.2 Management rule

- **Ghulam Qasim decides what must be delivered.**
- **Ghulam Qadir decides how modules are integrated.**
- **The named workstream lead decides implementation details inside the workstream.**
- Shared work does not mean shared editing: each file, endpoint, and contract still has one primary editor.
- No member changes another member's owned module without first creating an issue or obtaining agreement in the team channel.
- If a decision affects public contracts, security policy, deployment, or the demo, Ghulam Qasim and Ghulam Qadir must approve it.

1.2.1 Shared-workstream accountability

The requested group assignments are collaborative workstreams, not permission for everyone to edit the same files. The following technical leads prevent clashes:

Shared workstream	Technical lead	Contributors	Final decision
Backend and task orchestration	Azizullah	Yash, Jami	Qadir for integration; Qasim for scope
Security and policy	Asif	Azizullah, Qasim	Qasim for risk acceptance
Sandbox and tooling	Yash	Aamir, Asif	Asif for security; Qadir for integration
Context and model	Jami	Qadir, Aamir	Qadir for contract/integration
Validation, evaluation, documentation	Aamir	All members	Qasim for final acceptance

2. Recommended Technology Stack

The stack below is intentionally conservative so the team can finish in 25 days.

2.1 Core stack

Layer	Recommended technology	Reason
Language	Python 3.11+	Fast development, strong testing ecosystem, good AI/API support
API	FastAPI + Pydantic v2	Typed contracts, automatic OpenAPI, async support
Orchestration	Explicit Python state machine	Easy to inspect, test, and demonstrate
Database	PostgreSQL	Reliable run state, policies, approvals, metrics
Local development DB	SQLite only for lightweight unit tests	Avoid blocking local development
Queue	Redis + RQ/Celery, or an in-process MVP queue	Background run execution and cancellation
Cache	Redis	Context summaries, tool results, run locks
Repository index	SQLite/PostgreSQL + tree-sitter or Python AST	Faster MVP than introducing a complex vector platform
Semantic retrieval	Optional sentence-transformer/vector index	Add only after lexical/symbol retrieval works
Model provider	Nebius Token Factory OpenAI-compatible API	Required model integration
Model client	Official OpenAI-compatible Python client	Stable adapter interface
Sandbox	Docker/Podman rootless containers	Filesystem and process isolation
Validation	Pytest, Ruff, Mypy, Bandit where applicable	Tests, quality, type, and security checks

Layer	Recommended technology	Reason
Frontend/demo	Minimal React or server-rendered HTML	Keep UI small; prioritize working backend
Observability	Structured JSON logs + OpenTelemetry-compatible traces	Debugging and evaluation evidence
CI	GitHub Actions or equivalent	Automatic tests and quality checks
Documentation	Markdown + Mermaid diagrams	Version-controlled and easy to review

2.2 MVP scope rule

Do not add Kubernetes, microservices, a vector database, multiple frontend frameworks, or complex agent frameworks unless the basic vertical slice already works. The project is graded on a working, explainable system—not on the number of technologies used.

2.3 Suggested Python dependencies

```
fastapi
uvicorn
pydantic
pydantic-settings
httpx
openai
sqlalchemy
alembic
psycopg[binary]
redis
pytest
pytest-asyncio
ruff
mypy
bandit
python-dotenv
tree-sitter                # only if required for multi-language parsing
tenacity
orjson
opentelemetry-api          # optional but recommended
```

Pin dependency versions in `requirements.txt` or `pyproject.toml` . Do not install new packages directly into the shared environment without discussing the change first.

3. Ownership and Non-Conflict Boundaries

3.1 Directory ownership

The following directory ownership is mandatory. A member may review any directory, but the designated workstream lead remains the primary editor unless a specific file owner is assigned in the task board.

secure-coding-agent/	
— apps/	
— api/	Azizullah
— orchestrator/	Azizullah
— worker/	Azizullah + Yash + Jami
— packages/	
— contracts/	Ghulam Qadir
— task_analyzer/	Azizullah
— security_gateway/	Asif + Azizullah + Qasim
— prompt_injection/	Asif + Azizullah + Qasim
— context_optimizer/	Jami + Qadir + Aamir
— model_router/	Jami + Qadir + Aamir
— tool_router/	Yash + Aamir + Asif
— policy_engine/	Asif + Azizullah + Qasim
— sandbox_manager/	Yash + Aamir + Asif
— validator/	All members
— observability/	All members
— policies/	Asif + Azizullah + Qasim
— evaluation/	All members
— tests/	
— unit/	Each module owner
— integration/	Ghulam Qadir + Aamir
— security/	Asif + Azizullah + Qasim + Yash
— docs/	All members, coordinated by Aamir
— infra/	Yash + Aamir + Asif
— frontend/	Azizullah + Yash + Jami, with Qadir integrat:

3.2 Shared files requiring approval

Only Ghulam Qadir may merge changes to these files after review from the relevant owners:

```
pyproject.toml
requirements.txt / requirements.lock
.env.example
README.md
packages/contracts/*
Dockerfile / docker-compose.yml
CI workflow files
database migration registry
API route registration files
```

If a member needs a change in a shared file, they should:

1. Open an issue or task note.
2. Explain the required change and affected modules.
3. Let Ghulam Qadir make or coordinate the edit.
4. Run the full relevant test suite before merge.

3.3 Endpoint ownership registry

No two people may implement the same endpoint. The registry below is the source of truth.

Endpoint	Owner	Supporting owner	Status
POST /v1/runs	Azizullah	Qadir	Planned
GET /v1/runs/{run_id}	Azizullah	Qadir	Planned
POST /v1/runs/{run_id}/cancel	Azizullah	Yash	Planned
GET /v1/runs/{run_id}/events	Ghulam Qadir	Aamir	Planned
POST /v1/approvals/{approval_id}/decision	Asif	Azizullah	Planned
GET /v1/runs/{run_id}/diff	Aamir	Azizullah	Planned
GET /v1/runs/{run_id}/validation	Aamir	Azizullah	Planned
GET /v1/health	Ghulam Qadir	Azizullah	Planned
GET /v1/metrics	Ghulam Qadir	Aamir	Planned

Endpoint rule: An endpoint consists of its route, request schema, response schema, service method, database interaction, and tests. One primary owner is responsible for all five parts, even when the endpoint belongs to a shared workstream.

3.4 Contract-first rule

Before implementation, every cross-module interface must be written in `packages/contracts/` :

- Request and response models.
- Tool request and tool result schemas.
- Policy decision schema.
- Run state values.
- Error codes.
- Event names.
- Version number.

No member should silently alter a shared schema. A breaking change requires:

1. A short change note.
 2. Updated contract tests.
 3. Notification to all affected owners.
 4. Migration or compatibility handling.
 5. Approval from Ghulam Qadir.
-

4. Git and Branching Strategy

4.1 Branches

```
main                # always stable and demo-able
integration         # Qadir's integration branch
feature/<owner>/<task> # one branch per task
hotfix/<issue>       # urgent repair only
```

Examples:

```
feature/azizullah/task-api
feature/asif/policy-engine
feature/yash/tool-router
feature/jami/context-ranking
feature/aamir/validator
```

4.2 Rules that prevent conflicts

1. Never work directly on `main`.
2. Each branch has one clear task and one owner.
3. Keep branches short-lived; target one to three days maximum.
4. Rebase or merge from `integration` before requesting review.
5. Never force-push a shared branch.

6. Do not combine unrelated formatting changes with feature changes.
7. Avoid editing shared files unless the task explicitly requires it.
8. Push at least once per working day.
9. Open a pull request when the task is testable—not only when the whole module is finished.
10. Squash or clean noisy commits before merging.
11. Ghulam Qadir merges into `integration`.
12. Ghulam Qasim approves promotion from `integration` to `main`.

4.3 Commit format

```
<type>(<scope>): <short description>
```

Types: feat, fix, test, refactor, docs, security, infra, chore

Examples:

```
feat(api): add create-run endpoint
security(policy): deny credential paths by default
test validator): add structured pytest failure parsing
docs(plan): document approval workflow
```

4.4 Pull request template

Every PR must include:

```
## Purpose

## Files changed

## Contract/API impact

## Security impact

## Tests executed

## Reviewer requested

## Known limitations
```

4.5 Review matrix

Change area	Required reviewer
API/orchestrator	Qadir + Azizullah
Security/policy	Asif + Yash + Qadir
Sandbox/tool execution	Yash + Asif + Qadir
Context/model	Jami + Aamir + Qadir
Validator/evaluation	Aamir + Qadir
Shared contracts	Qadir + affected module owner
Final release	Qasim + Qadir + all owners

5. Team Operating Rhythm

5.1 Daily schedule

Use one short daily meeting or written stand-up. Maximum 15 minutes.

Each member reports:

1. What I completed yesterday.
2. What I will complete today.
3. What is blocked.
4. Which files or contracts I need from another member.

The stand-up is not for long technical debates. Move debates to a short design note or a separate 20-minute session.

5.2 Daily time blocks

Block	Activity
Start of day	Review task board, contracts, and integration status
Main work block	Individual implementation in owned branch
Midday	Push progress and report blockers
Integration block	Qadir merges tested work and runs CI
End of day	Update task status, test evidence, and next action

5.3 Task board columns

Backlog → Ready → In Progress → Review → Integration → QA → Done

A task cannot enter **In Progress** unless it has:

- One owner.
- A clear acceptance criterion.
- Expected files/directories.
- Dependencies identified.
- A target completion date.

5.4 Conflict-resolution protocol

When members disagree:

1. State the issue in one sentence.
2. Identify whether it affects scope, contract, security, or implementation detail.
3. Prefer the smallest change that preserves the agreed contract.
4. If security is involved, Asif may block the merge until the risk is resolved.
5. If integration is involved, Qadir decides the merge sequence.
6. If scope or deadline is involved, Qasim decides.
7. Record the decision in `docs/decisions.md`.

No disagreement should remain unresolved for more than one working day.

6. Work Distribution by Member

6.1 Ghulam Qasim — Team Lead / Product Owner

Responsibilities

- Freeze MVP scope on Day 1.
- Approve architecture and acceptance criteria.
- Prioritize tasks and remove non-essential features.
- Review progress and resolve scope conflicts.
- Conduct stakeholder communication.
- Approve final release and submission.

Deliverables

- Product scope and definition of done.
- Final feature priority list.
- Acceptance criteria for the demo.
- Final project review checklist.
- Presentation/demo narrative.

Must not do

- Do not become the bottleneck by manually reviewing every file.
- Do not make frequent direct edits inside member-owned modules.
- Do not change technical contracts informally.

Efficient review method

Qasim should review:

- PR summaries and changed-file lists.
- Security decisions and blocked actions.
- CI results and integration status.
- Final API behavior and demo flow.
- High-risk architectural changes.

Qadir performs the detailed code review so Qasim does not need to inspect every line.

6.2 Ghulam Qadir — Co-Lead / Integration Lead

Responsibilities

- Create repository skeleton and branch protections.
- Own shared contracts and integration branch.
- Maintain endpoint registry and dependency map.
- Run daily integration builds.
- Review cross-module changes.
- Coordinate release candidates.
- Maintain CI and shared development scripts.

Deliverables

- Repository layout.
- `packages/contracts/` schemas.
- Integration branch and CI pipeline.

- Run event stream and health endpoints.
- Integration test suite.
- Release checklist.

Authority

Qadir can reject a PR for:

- Contract mismatch.
 - Missing tests.
 - Unclear ownership.
 - Shared-file collision.
 - Unresolved dependency.
 - Broken integration branch.
-

6.3 Azizullah + Yash + Jami — Backend and Task Orchestration Workstream

Workstream lead: Azizullah

Execution integration: Yash

Context/model integration: Jami

Owns

```
apps/api/  
apps/orchestrator/  
packages/task_analyzer/  
frontend/ basic task screens
```

Work items

- FastAPI application setup.
- Create-run, status, cancel, and basic run retrieval endpoints.
- Run lifecycle state machine.
- Task input validation.

- Repository/task metadata persistence integration.
- Worker invocation interface.
- Task analyzer and structured task object.
- Backend unit and API tests.

Dependencies

- Contracts from Qadir.
- Policy decisions from Asif.
- Sandbox execution interface from Yash.
- Model worker interface from Jami.

Handoff requirements

Before handing off, provide:

- OpenAPI output.
 - Example request/response payloads.
 - State transition tests.
 - Error behavior.
 - Database migration instructions.
-

6.4 Asif + Azizullah + Ghulam Qasim — Security and Policy Workstream

Workstream lead: Asif

Orchestration integration: Azizullah

Risk acceptance and final approval: Ghulam Qasim

Owns

```
packages/security_gateway/  
packages/prompt_injection/  
packages/policy_engine/  
policies/  
tests/security/
```

Work items

- Filesystem allow/deny policy.
- Shell risk classification.
- Network destination policy.
- Git action policy.
- Tool authorization decisions.
- Approval state and high-risk action handling.
- Prompt-injection labeling and detection.
- Secret redaction utilities.
- Audit event definitions.
- Security attack test suite.

Dependencies

- Tool schemas from Qadir.
- Runtime enforcement hooks from Yash.
- Run identity from Azizullah.

Non-negotiable rules

- Security policy must be deterministic for known inputs.
 - Unknown capabilities must default to deny.
 - The model must never be treated as an authorization source.
 - Security tests must run before every release candidate.
-

6.5 Yash + Aamir + Asif — Sandbox and Tooling Workstream

Workstream lead: Yash

Validation and evidence: Aamir

Security review: Asif

Owns

```
packages/tool_router/  
packages/sandbox_manager/  
infra/container/  
infra/runtime/
```

Work items

- Typed tool router.
- Repository read/write tools.
- Structured shell execution.
- Test runner adapter.
- Git read-only adapter.
- Rootless sandbox/container setup.
- Mount restrictions and path canonicalization hooks.
- CPU, memory, disk, process, and timeout limits.
- Runtime output capture.
- Network proxy integration point.

Dependencies

- Policy decisions from Asif.
- Tool contracts from Qadir.
- Run lifecycle from Azizullah.

Handoff requirements

Demonstrate that the runtime blocks:

- `~/ .ssh` and credential paths.
 - `../` path traversal.
 - Unrelated repositories.
 - Dangerous commands.
 - Unapproved network destinations.
 - Host process or filesystem access.
-

6.6 Jami + Ghulam Qadir + Aamir — Context and Model Engineering Workstream

Workstream lead: Jami

Contract and integration: Ghulam Qadir

Evaluation: Aamir

Owns

```
packages/context_optimizer/  
packages/model_router/
```

Work items

- Repository file inventory.
- Symbol/import/reference index.
- Relevance scoring.
- Context budget allocation.
- Context compression.
- Context cache keys and invalidation.
- Tool-result summarization.
- Loop detection record and threshold logic.
- Nebius OpenAI-compatible adapter.
- Model selection and parameter configuration.
- Token and latency measurement.

Dependencies

- Task object from Azizullah.
- Tool result schema from Qadir.
- Runtime output from Yash.
- Evaluation tasks and metrics from Aamir.

Handoff requirements

Provide comparison evidence for:

- Full repository context vs indexed context.
- Uncompressed output vs compressed output.
- At least two models or two routing configurations.
- Token count, latency, tool calls, and task success.

6.7 All Members — Validation, Evaluation, and Documentation Workstream

Workstream coordinator: Aamir

Final acceptance: Ghulam Qasim

Integration control: Ghulam Qadir

Owns

```
packages/validator/  
evaluation/  
docs/  
tests/integration/
```

Work items

- Patch/diff validator.
- Syntax, formatting, lint, and test validation pipeline.
- Structured test failure parser.

- Critic result schema.
- Functional task suite.
- Security evaluation suite in coordination with Asif and Yash.
- Baseline A/B/C experiment harness.
- Metrics report.
- User guide, setup guide, architecture summary, and final report.
- Demo script and expected outputs.

Dependencies

- Run results from Azizullah.
- Policy/audit results from Asif.
- Sandbox output from Yash.
- Context/model metrics from Jami.

Handoff requirements

Every validation report must show:

- What was tested.
 - Exact command.
 - Pass/fail result.
 - Relevant failure evidence.
 - Security findings.
 - Recommendation.
-

7. 25-Day Delivery Schedule

The schedule is divided into five phases. Every phase ends with a review gate. The final five days are protected for integration, bug fixing, documentation, and submission; new major features are prohibited during that period.

Phase overview

Phase	Days	Goal	Gate
Phase 1	1–4	Scope, contracts, skeleton, environment	Architecture and contract freeze
Phase 2	5–10	Build independent modules	Each module has unit tests
Phase 3	11–16	First vertical integration	End-to-end happy path works
Phase 4	17–20	Security, evaluation, and optimization	Release candidate 1
Phase 5	21–25	Stabilize, document, rehearse, submit	Final release

Day-by-Day Plan

Day 1 — Kickoff and scope freeze

All members

- Review architecture and this work distribution.
- Confirm the MVP feature list.
- Confirm what is explicitly out of scope.
- Install the agreed toolchain.
- Create communication channel and task board.

Qasim

- Approve the definition of done.
- Select the demonstration scenario.

Qadir

- Create repository, branches, owners, issue labels, PR template, and basic CI.